

5.1 - Beginner JavaScript Problems

1. Multiply Two Numbers

Objective: Write a function that will receive 2 numbers as input and it should return the multiplication of these 2 numbers.

Input: Two numbers.

Output: Number.

Example:

```
multiplyTwoNumbers(2, 3) == 6  
multiplyTwoNumbers(1, 0) == 0
```

2. Say Hi

Objective: In this mission you should write a function that introduces a person with the given parameter's attributes.

Input: Two arguments. String and positive integer.

Output: String.

Example:

```
sayHi("Alice", 32) == "Hi. My name is Alice and I'm 32 years old."  
sayHi("Bob", 68) == "Hi. My name is Bob and I'm 68 years old."
```

3. Index Power

Objective: You are given an array with positive numbers and a number N. You should find the N-th power of the element in the array with the index N. If N is outside of the array, then return -1. Don't forget that the first element has the index 0.

Let's look at a few examples:

- `array = [1, 2, 3, 4]` and `N = 2`, then the result is `32 == 9`;
- `array = [1, 2, 3]` and `N = 3`, but N is outside of the array, so the result is `-1`.

Input: Two arguments. An array as a list of integers and a number as a integer.

Output: The result as an integer.

Example:

```
indexPower([1, 2, 3, 4], 2) == 9
indexPower([1, 3, 10, 100], 3) == 1000000
indexPower([0, 1], 0) == 1
indexPower([1, 2], 3) == -1
```

4. Fizz Buzz

Objective: You should write a function that will receive a positive integer and return:

Fizz Buzz if the number is divisible by **3** and by **5** ; **Fizz** if the number is divisible by **3** ; **Buzz** if the number is divisible by **5** ; The number as a string for other cases.

Input: A number as an integer.

Output: The answer as a string.

Example:

```
fizzBuzz(15) == "Fizz Buzz"
fizzBuzz(6) == "Fizz"
fizzBuzz(5) == "Buzz"
fizzBuzz(7) == "7"
```

5. Digits Multiplication

Objective: You are given a positive integer. Your function should calculate the product of the digits excluding any zeroes.

The number given is **123405** . The result will be **1*2*3*4*5=120** (don't forget to exclude zeroes).

Input: A positive integer.

Output: The product of the digits as an integer.

Example:

```
multiplyDigits(123405) == 120
multiplyDigits(999) == 729
multiplyDigits(1000) == 1
multiplyDigits(1111) == 1
```

6. Secret Code

Objective: You are given a chunk of text. Gather all capital letters in one word in the order that they appear in the text.

Input: A text as a string (unicode).

Output: The secret message as a string or an empty string.

Example:

```
findMessage("How are you? Eh, ok. Low or Lower? Ohhh.") == "HELLO"
findMessage("hello world!") == ""
```

7. The Most Frequent

Objective: You have a sequence of strings, and you'd like to determine the most frequently occurring string in the sequence. It can be only one.

Input: non empty Array of strings.

Output: a string.

Example:

```
mostFrequent(['a', 'b', 'c', 'a', 'b', 'a']) == 'a'
mostFrequent(['hi', 'hi', 'bye', 'bye', 'bye']) == 'bye'
```

8. Non-Unique Elements

Objective: You are given a non-empty list of integers (X). For this task, you should return a list consisting of only the non-unique elements in this list. To do so you will need to remove all unique elements (elements which are contained in a given list only once). When solving this task, do not change the order of the list. Example: `[1, 2, 3, 1, 3]` `1` and `3` non-unique elements and result will be `[1, 3, 1, 3]`.

Input: A list of integers.

Output: An iterable of integers.

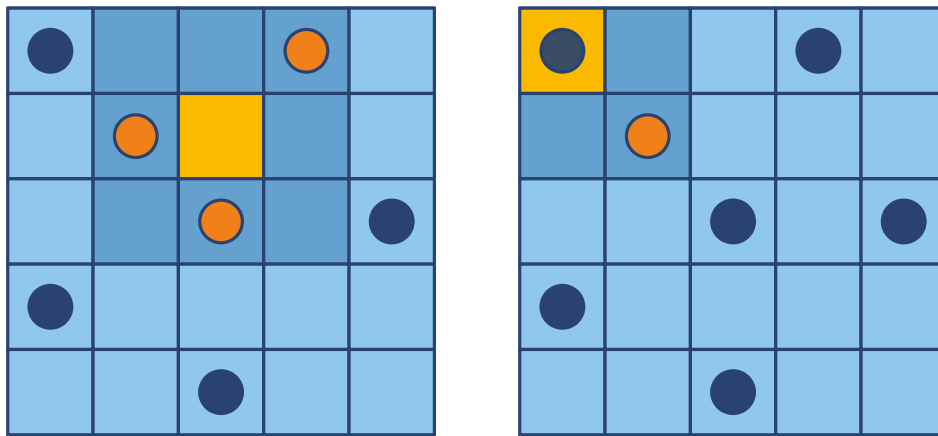
Example:

```
nonUniqueElements([1, 2, 3, 1, 3]) == [1, 3, 1, 3]
nonUniqueElements([1, 2, 3, 4, 5]) == []
nonUniqueElements([5, 5, 5, 5, 5]) == [5, 5, 5, 5, 5]
nonUniqueElements([10, 9, 10, 10, 9, 8]) == [10, 9, 10, 9]
```

9. Moore's Neighbourhood

In cellular automata, the *Moore Neighborhood* comprises the eight cells surrounding a central cell on a two-dimensional square lattice. The neighborhood is named after Edward F. Moore, a pioneer of cellular automata theory. Many board games are played with a rectangular grid with squares as cells. For some games, it is important to know about the conditions of neighbouring cells for chip (figure, draught etc) placement and strategy.

Objective: You are given a state for a rectangular board game grid with chips in a binary matrix, where 1 is a cell with a chip and 0 is an empty cell. You are also given the coordinates for a cell in the form of row and column numbers (starting from 0). You should determine how many chips are close to this cell. Every cell interacts with its eight neighbours; those cells that are horizontally, vertically, or diagonally adjacent.



For the given examples (see the schema) there is the same grid:

```
[[1, 0, 0, 1, 0],  
 [0, 1, 0, 0, 0],  
 [0, 0, 1, 0, 1],  
 [1, 0, 0, 0, 0],  
 [0, 0, 1, 0, 0]]
```

For the first example coordinates of the cell is (1, 2) and as we can see from the schema this cell has 3 neighbour chips. For the second example coordinates is (0, 0) and this cell contains a chip, but we count only neighbours and the answer is 1.

Input: Three arguments. A grid as a tuple of tuples with integers (1 or 0), a row number and column number for a cell as integers.

Output: How many neighbouring cells have chips as an integer.

Example:

```
countNeighbours([[1, 0, 0, 1, 0],  
                [0, 1, 0, 0, 0],  
                [0, 0, 1, 0, 1],  
                [1, 0, 0, 0, 0],  
                [0, 0, 1, 0, 0]], 1, 2) == 3
```

```
countNeighbours([[1, 0, 0, 1, 0],  
                [0, 1, 0, 0, 0],  
                [0, 0, 1, 0, 1],  
                [1, 0, 0, 0, 0],  
                [0, 0, 1, 0, 0]], 0, 0) == 1
```